

Onboarding & Handoff

Getting started and current roadmap

Document	8 of 9
Audience	New team members
Revision	Rev 1.0 · 2026
Status	Handoff

Castillo Planset QC — Onboarding & Handoff Guide

Audience: a new Developer or Engineer joining the Castillo Planset QC project. This is the definitive "day one to first deploy" reference. It assumes you have working knowledge of Python, TypeScript/React, and basic Azure concepts, but no prior exposure to this codebase.

1. What you are inheriting

Castillo Planset QC is an AI-assisted QA/QC tool for solar PV plansets. An engineer uploads a planset PDF; the system runs a mix of deterministic NEC electrical calculations, keyword/sheet-existence/title-block rules, and vision-model checks, then produces a triaged list of findings (Pass / Fail / Needs Review / Deferred) with severity, confidence, evidence, and page references that a QC Engineer can review, adjust, and export to Excel.

Shape of the system:

Layer	Technology	Location
Backend API + pipeline	FastAPI (Python 3.11)	<code>backend/app/</code>
Frontend SPA	React 18 + Vite + TypeScript	<code>frontend/src/</code>
AI provider abstraction	OpenAI (default) / Gemini	<code>backend/app/gemini_client.py</code>
Persistence	SQLite on an Azure Files share	<code>/home/data</code> in prod; <code>backend/data/</code> locally
Hosting	Azure Container Apps (single Linux container: FastAPI serves both API and built SPA)	—
CI/CD	GitHub Actions + OIDC + <code>az acr build</code>	<code>.github/workflows/</code>

In production the FastAPI backend serves the compiled React bundle itself, so the API is same-origin. Only during local Vite development are the two served separately.

2. Prerequisites

Before you can build, run, and deploy, arrange the following.

2.1 Local toolchain

Tool	Version	Notes
------	---------	-------

Python	3.11.x	Matches the <code>python:3.11-slim</code> runtime image. Use a virtual environment.
Node.js	22.x	Matches the <code>node:22-slim</code> build image used by CI.
npm	Ships with Node 22	CI uses <code>npm ci</code> , so a committed <code>frontend/package-lock.json</code> is required.
Azure CLI (<code>az</code>)	Recent	For inspecting/operating the Container Apps, ACR, and Azure Files.
GitHub CLI (<code>gh</code>)	Recent	For PRs and workflow runs.
Git	Recent	—

2.2 Access to request from the DevOps/Automation Engineer

You will need all of the following before you can deploy or debug production:

- **Microsoft Entra tenant** — membership in the single-tenant `castillope.com` org directory. EasyAuth restricts the app to organization sign-in, so you cannot even reach the deployed UI without a tenant account.
- **Azure subscription** — Contributor (or scoped) access to the two resource groups: `castillo-qaqc-automation-rg` (prod) and the sandbox environment.
- **ACR pull/build** — access to the Azure Container Registry `castilloqaqcautomationacr`. Production image pulls use a **managed identity**; you only need registry access for manual builds/inspection.
- **GitHub repository** — write access, plus visibility into the repository Actions secrets (values themselves are held by the DevOps/Automation Engineer; never copy them locally).
- **An AI provider API key** for local runs — an OpenAI key (default provider) or a Gemini key. **Never** commit this; it goes only in your gitignored `backend/.env`.

Prod app: `castillo-qaqc-automation` in RG `castillo-qaqc-automation-rg`. Sandbox app: `castillo-qaqc-sandbox`. Both run one always-on replica with an Azure Files share mounted at `/home/data`.

3. First-time local setup

3.1 Clone

```
git clone <repo-url> planset-qc-app
cd planset-qc-app
```

3.2 Configure the backend environment

Copy the template and fill in a real AI key. `backend/.env` is gitignored — keep it that way.

```
cp backend/.env.example backend/.env
```

Then edit `backend/.env`. The meaningful variables (defaults shown as read by `backend/app/gemini_client.py`, `db.py`, and `jobs.py`):

Variable	Default	Purpose
AI_PROVIDER	openai	openai or gemini.
OPENAI_API_KEY	—	Required when provider is openai . Secret.
OPENAI_MODEL	gpt-4.1-mini	Per-page / lighter passes.
OPENAI_MODEL_DEEP	falls back to OPENAI_MODEL	Heavier "deep" passes.
OPENAI_SEED	unset	Optional integer for best-effort determinism across runs (clean before/after diffs). Invalid values are ignored, not fatal.
GEMINI_API_KEY	—	Required when provider is gemini . Secret.
GEMINI_MODEL	gemini-2.5-flash	—
GEMINI_MODEL_DEEP	falls back to GEMINI_MODEL	—
AI_PARALLELISM	6 (tuning)	Optional AI concurrency tuning.
AI_TIMEOUT	90	Per-request AI timeout (seconds).
AI_RETRIES	2	AI retry count.
ANALYZE_CONCURRENCY	2	Bounded analysis job-queue worker count (<code>jobs.py</code>).
PLANSET_DATA_DIR	backend/data locally; /home/data in Azure	Runs, SQLite, uploads, exports, logs. Leave unset locally.
DEV_USER_EMAIL	unset	Local-only identity fallback so signed-in attribution works without

		EasyAuth headers.
<code>RULES_FILE</code>	app default	Selects the rules file (the launcher and prod use <code>rules_v4_draft.yaml</code>).
<code>FRONTEND_DIST</code>	unset locally; set in container	When set, FastAPI mounts the built SPA. Leave unset for split local dev.

Never place real keys, connection strings, or the Azure Files secret in any committed file. In Azure these values are Container App application settings managed by the DevOps/Automation Engineer.

3.3 Install and run the backend

```
cd backend
python -m venv .venv
# Windows PowerShell: .venv\Scripts\Activate.ps1
# bash:                source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Backend deps (`backend/requirements.txt`): `fastapi`, `uvicorn[standard]`, `python-multipart`, `PyMuPDF`, `pdfplumber`, `openpyxl`, `Pillow`, `google-genai`, `python-dotenv`, `PyYAML`, `openai`.

Confirm it is up: `GET http://localhost:8000/api/health`.

3.4 Install and run the frontend

In a second terminal:

```
cd frontend
npm ci          # or: npm install
npm run dev     # Vite dev server on :5173
```

The frontend automatically talks to the backend on port 8000 when it detects it is being served from Vite's port 5173 (see `frontend/src/App.tsx`: `API = window.location.port === "5173" ? http://<host>:8000 : ""`). No proxy configuration is needed.

Open `http://localhost:5173/`.

3.5 One-command Windows launcher (optional)

`start.bat` at the repo root frees ports 8000/5173, starts the backend with `RULES_FILE=rules_v4_draft.yaml`, and starts the Vite dev server. Useful for demos on the office LAN.

3.6 Local identity

Locally there is no Entra/EasyAuth header, so signed-in attribution falls back to `DEV_USER_EMAIL` (see `backend/app/auth.py`). Set it in `backend/.env` if you want your runs attributed while developing; otherwise the legacy free-text engineer-name field is the last-resort display fallback.

4. Backend API surface (orientation)

You will spend most of your time in `backend/app/main.py`. The routes, at a glance:

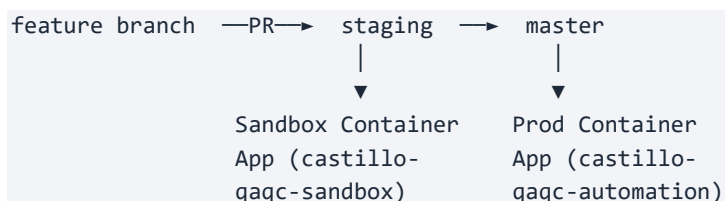
GET	/api/health	
GET	/api/me	# resolved signed-in identity
GET	/api/runs	POST /api/analyze
GET	/api/runs/{run_id}	DELETE /api/runs/{run_id}
GET	/api/runs/{run_id}/versions	POST /api/runs/{run_id}/reanalyze # non-destructive
GET	/api/projects	GET /api/projects/{project_id}
GET	/api/jobs	# shared Activity panel feed
GET	/api/progress/{upload_id}	GET /api/result/{upload_id}
POST	/api/parse-project-details	POST /api/parse-supporting-docs
POST	/api/issues/manual	POST /api/issues/{issue_id}/feedback
POST	/api/runs/{run_id}/feedback	GET /api/runs/{run_id}/feedback
GET	/api/export/{run_id}	GET /api/due-diligence-template

Pipeline modules worth knowing early: `analyzer.py` (orchestration), `electrical_calcs.py` (deterministic NEC math), `rules.yaml` / `rules_v4_draft.yaml` + `rule_registry.py` / `v4_engine.py` (rule engine), `consistency.py` and `diagram_consistency.py` (cross-sheet checks), `supporting_docs.py` (tech-spec cross-checks), `dedup_findings.py` (finding de-duplication), `exporter.py` (XLSX), `jobs.py` (bounded job queue), `db.py` (SQLite + projects/versioning schema), `auth.py` (EasyAuth identity resolver).

Status color convention used throughout the UI: **Pass = green, Fail = red, Needs Review = amber, Deferred = purple.**

5. Git branch & deploy model

Three-tier promotion; **weekday deploys only** (see §8 on cadence):



1. **Feature branch** — branch off `master`, do your work, open a PR. CI does not auto-deploy feature branches.
2. ``staging`` → **Sandbox** — merging/pushing to `staging` triggers `.github/workflows/deploy-staging.yml`. It builds `planset-qc:staging-<sha>` in ACR and rolls the **sandbox** app (`castillo-qaqc-sandbox`). Try experimental versions here without touching production. Staging image tags are prefixed `staging-` so they never collide with prod tags.
3. ``master`` → **Prod** — merging to `master` triggers `.github/workflows/deploy.yml`. It builds `planset-qc:<sha>` and `planset-qc:latest` in ACR and rolls the **prod** app.

Both workflows authenticate with Azure via **OIDC federated credentials** (no stored passwords/publish profiles). The OIDC app trusts `refs/heads/master` and `refs/heads/staging`. They share the same GitHub Actions secrets: `AZURE_CLIENT_ID`, `AZURE_TENANT_ID`, `AZURE_SUBSCRIPTION_ID`, `AZURE_RESOURCE_GROUP`, `AZURE_CONTAINERAPP_NAME`, `ACR_NAME` (the sandbox app name is inlined in the staging workflow, as it is not sensitive). Both can also be launched manually via ``workflow_dispatch``.

The container is built by the multi-stage `Dockerfile` (stage 1: `npm ci && npm run build`; stage 2: Python backend + bundled SPA at `/app/frontend_dist`, with `FRONTEND_DIST`, `PLANSET_DATA_DIR=/home/data`, `AI_PROVIDER=openai` baked in). Ingress targets port 8000.

Full infrastructure details, secret setup, and the ACR/managed-identity story live in `DEPLOYMENT.md` and `infra/main.bicep`. Container Apps (not App Service) is used because the subscription has zero App Service VM quota.

6. Running the test & regression scripts

Everything under `backend/scripts/` is runnable standalone. Run from the `backend/` directory (scripts insert the backend root on `sys.path`), for example:

```
cd backend
python scripts/test_projects_versioning.py
# some scripts document: PYTHONPATH=backend python backend/scripts/<name>.py
```

Most tests run **without any AI/network call** against a throwaway temp SQLite DB.

Functional / unit tests (no network):

Script	Covers
<code>test_projects_versioning.py</code>	Projects/stage/multi-user data layer, legacy backfill, non-destructive rerun versioning, and the EasyAuth

	identity resolver header / base64-claims / dev-fallback paths.
<code>test_jobs.py</code>	Bounded job queue: concurrency cap, queued→running→done/error transitions, queue positions, error handling, activity-feed metadata.
<code>test_seed_option.py</code>	<code>OPENAI_SEED</code> helper logic (unset → {}, 42 → {seed:42}, bad value → ignored) plus a live probe of model acceptance.
<code>test_consistency.py</code> , <code>test_diagram_consistency.py</code>	Cross-sheet / diagram consistency checks.
<code>test_dedup_findings.py</code>	Finding de-duplication.
<code>test_page_targeting.py</code> , <code>test_referent_verification.py</code>	Page-targeting and referent verification.
<code>test_stage_gating.py</code> , <code>test_prompt_fixes.py</code>	Stage gating and prompt regression fixes.

Regression harnesses (heavier; some hit ground-truth data / cached runs):

Script	Purpose
<code>regression_v4.py</code>	V4 engine regression against hand-configured pilots.
<code>regression_all_plansets.py</code>	Scales the harness across all matched xlsx+PDF pairs; emits scorecard/ground-truth workbooks under <code>backend/data/regression/</code> .
<code>regression_snapshot.py</code> + <code>regression_baseline.json</code>	Snapshot-vs-baseline comparison.
<code>diagnose_wellington_nr.py</code> , <code>verify_tuning_bishop.py</code> , <code>compare_gonzo_findings.py</code> , <code>bucket_gonzo_*.py</code>	Targeted per-planset diagnostics.
<code>ingest_v4_checklist.py</code> , <code>extract_stage_sheets.py</code> , <code>tag_training_docs.py</code> , <code>merge_tag_batches.py</code>	Data-prep / ingestion utilities.

Before shipping anything that touches the rules engine or pipeline, run the relevant `test_*.py` and, for engine changes, a regression pass. See `TESTING_PLAN.md`.

7. The review / QC loop

This is a human-in-the-loop tool. Two roles gate what ships:

- **The QC Engineer** is the reviewer of record. The tool's findings are a starting point, not the final word. They run plansets through the app, mark findings (accept, override, correct), and give structured feedback on which rules are accurate, noisy, or wrong. Their feedback drives which rules stay, get tuned, or get deleted. Their working reference is `QC_ENGINEER_GUIDE.md`.
- **The Engineer of Record (PE)** signs off on anything that changes an engineering judgment or an NEC interpretation. Code that alters how a calculation or code rule is applied does **not** merge to `master` on developer confidence alone — it carries a **PE gate**.

Practical loop for a Developer:

4. Take a QC-feedback item (a false positive, a missed check, a noisy rule).
5. Branch, implement, add/extend a `backend/scripts/test_*.py`, and run the regression harness if the rules engine is touched.
6. Push to `staging`, verify on the `sandbox` app with the QC Engineer.
7. For anything touching NEC/engineering judgment, obtain **PE sign-off** before merging to `master`.
8. Merge to `master`; confirm the prod rollout and re-verify on a representative planset.

8. Working cadence

All scheduled work, deploys, reviews, and any on-call expectations are **Monday–Friday only**. Do not plan weekend deploy windows, weekend verification passes, or weekend on-call rotations. Time promotions to `master` for a weekday when the QC Engineer and, where required, the PE are available to verify.

9. Current state & roadmap

The active workstream is remediating QC-Engineer feedback in waves.

- **Wave 1 — shipped to an open PR (PE gate pending)**. Prompt/rule false-positive fixes and the `OPENAI_SEED` best-effort determinism option are implemented and in an open pull request. The **MCOV change in this wave carries a PE gate** — it must not merge to `master` until the Engineer of Record signs off.
- **Wave 2 — next**. Three items: (a) a **notation-consistency guard**, (b) a **tech-spec-to-Issues mirror** (surface tech-spec cross-check results into the Issues surface), and (c) finishing/rolling out the **seed option**.
- **Wave 3 — needs PE sign-off**. **PAD sheet-routing** corrections and **grounding-extraction** fixes. Because these change engineering interpretation, they require PE review before merge.

- **NEC-edition selector — designed, not built.** An engineer-selectable NEC code year (2017 / 2020 / 2023 / 2026), pinned per project, has an approved design but no implementation yet.

When picking up any wave item, confirm current status via the open PR and the workflow runs before starting — do not assume this list is fresh.

10. Key ownership by role

Area	Owning role
Backend pipeline, rules engine, API, frontend SPA, tests	the Developer
Azure infra, Container Apps, ACR, OIDC/secrets, CI/CD workflows, Entra/EasyAuth config, Azure Files/SQLite share	the DevOps/Automation Engineer
Rule accuracy, triage feedback, acceptance of the tool's output on real plansets	the QC Engineer
NEC interpretation, engineering-judgment sign-off, PE-gated changes (e.g., MCOV, grounding, PAD routing, NEC edition)	the Engineer of Record (PE)

Access requests, secret rotation, and production operations route through **the DevOps/Automation Engineer**. Never self-provision secrets or copy Actions secret values locally.

11. Where the other handoff documents live

All at the repository root unless noted:

Document	Contents
README.md	Product overview, what the pipeline does, the V4 rule engine, cross-sheet consistency.
DEPLOYMENT.md	Azure architecture, Container Apps, ACR, OIDC secrets, managed-identity pull, Azure Files mount.
QC_ENGINEER_GUIDE.md	The QC Engineer's operating guide and the all-important feedback section.
USER_GUIDE.md	End-user walkthrough of the app.
TESTING_PLAN.md	Test and regression strategy.

<code>docs/RULES_ENGINE_AUDIT.md</code>	Deep audit of the rules engine.
<code>infra/main.bicep</code> , <code>infra/main.parameters.json</code>	Infrastructure-as-code for the Azure resources.

12. First-week checklist

9. Get Entra tenant access and confirm you can sign in to the deployed **sandbox** app.
10. Clone, create `backend/.env` from `backend/.env.example`, add an AI key, run backend + frontend locally, and process a sample planset end to end.
11. Run `python scripts/test_projects_versioning.py` and `python scripts/test_jobs.py` from `backend/` and confirm they pass.
12. Read `README.md`, `DEPLOYMENT.md`, and `QC_ENGINEER_GUIDE.md`.
13. Ship a trivial change through the full loop: feature branch → PR → **staging** → verify on sandbox with the QC Engineer. Do **not** touch **master** or any PE-gated area until you have walked the loop once.